



Bitcoin & Stock Simulation Portfolio

1. 프로젝트 개요 (Overview)

Bitcoin & Stock Simulation은 암호화폐(Bitcoin)와 미국 주식 시장의 과거 데이터를 기반으로 다양한 매매 전략을 검증(Backtesting)하고, 실시간 시장 데이터를 분석하여 자동 매매를 수행할 수 있는 웹 기반 시뮬레이션 플랫폼입니다.

단순한 차트 뷰어를 넘어, **AI 모델(TimesFM, FinBERT)**을 활용한 가격 예측 및 뉴스 감성 분석 기능을 통합하여 투자 의사결정을 지원하며, **한국투자증권(KIS) API**와 연동하여 실제 계좌의 자산 관리 및 자동 매매 기능까지 확장된 올인원 트레이딩 솔루션입니다.

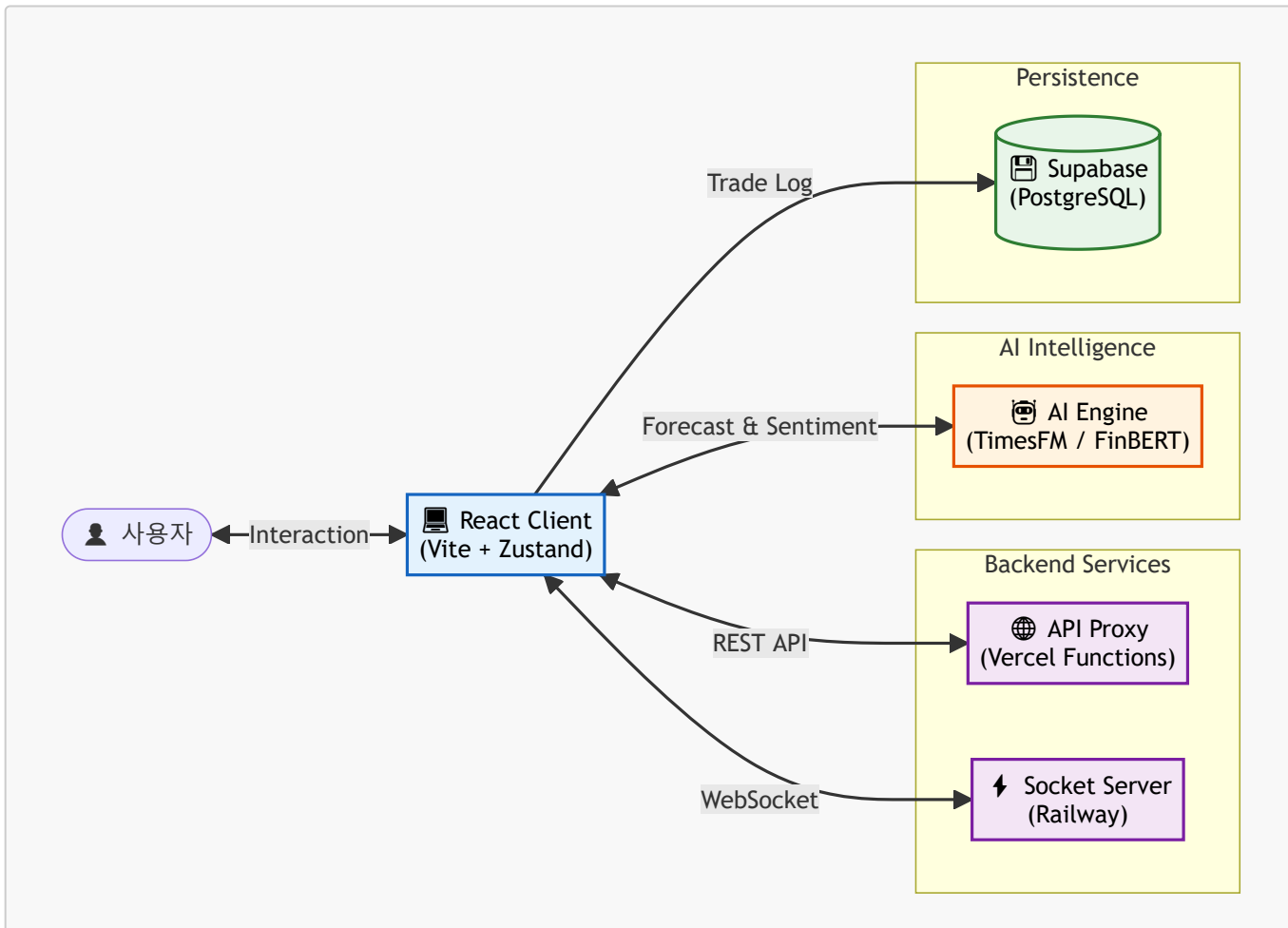
2. 시스템 아키텍처 (System Architecture)

본 프로젝트는 확장성과 유지보수성을 고려하여 **Frontend(UI) - Serverless(Proxy) - AI/Data Services**가 분리된 구조로 설계되었습니다.

📊 시스템 구조 및 흐름 (Architecture & Flow)

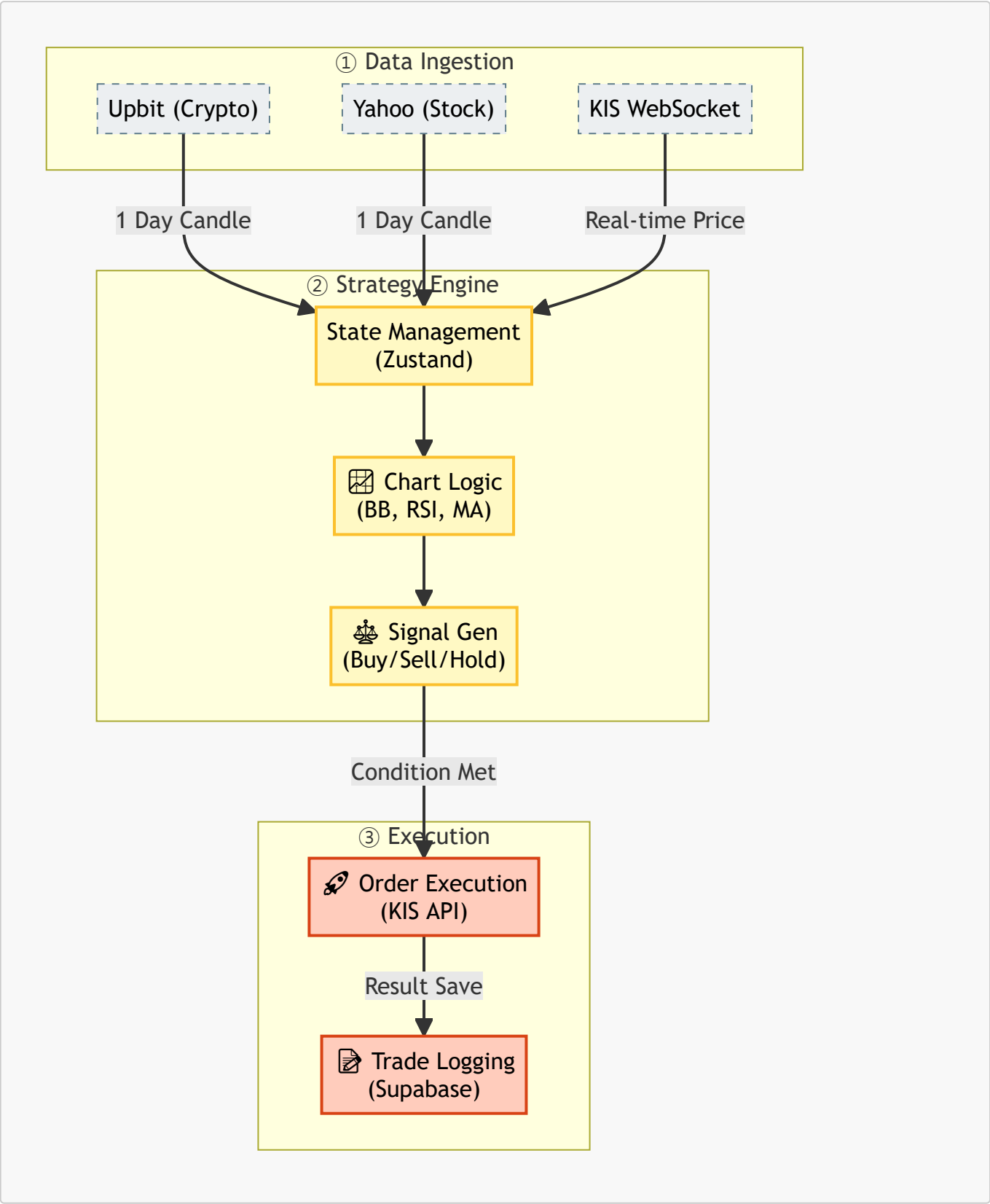
1. 하이레벨 아키텍처 (High-Level Architecture)

React 프론트엔드를 중심으로 백엔드 서비스, AI 엔진, 데이터베이스가 어떻게 유기적으로 연결되는지 보여줍니다.



2. 데이터 처리 및 매매 파이프라인 (Data & Trading Pipeline)

외부 데이터 수집부터 전략 분석, 그리고 실제 매매 주문이 실행되기까지의 상세 흐름입니다.



🌐 외부 서버 및 인프라

- **Hosting:** Vercel (Frontend & Serverless Functions)

- **AI Backend:** **Hugging Face Spaces** (Google TimesFM 모델 서빙, Python/FastAPI)
- **Real-time Server:** **Railway** (Node.js/WebSocket 중계 서버)
- **Database:** **Supabase** (PostgreSQL - 매매 이력 영구 저장)

3. 기술 스택 및 선정 이유 (Tech Stack & Decision)

단순히 최신 기술을 사용하는 것을 넘어, 프로젝트의 요구사항에 가장 적합한 도구를 선택했습니다.

Frontend

기술	선정 이유 (Why?)
React 19 (Vite)	VDOM 기반의 효율적인 렌더링과 방대한 생태계. Vite를 통해 HMR 속도를 극대화하여 개발 생산성을 높였습니다.
Zustand	Redux의 높은 보일러플레이트 복잡도를 줄이고, Context API의 불필요한 리렌더링 문제를 피하기 위해 Atomic하고 직관적인 상태 관리 라이브러리를 선택했습니다.
React Query	API 데이터 캐싱, 자동 재요청, 로딩 상태 관리를 위해 사용. 서버 상태(Server State)와 클라이언트 상태(Client State)를 명확히 분리했습니다.
Tailwind CSS + shadcn/ui	유틸리티 퍼스트 CSS로 스타일링 시간을 단축하고, 접근성(A11y)이 보장된 shadcn/ui 컴포넌트로 완성도 높은 UI를 빠르게 구축했습니다.
Recharts	React 컴포넌트 친화적이며 커스터마이징이 용이하여, 복잡한 캔들스틱 차트와 보조지표 레이어를 구현하는 데 최적화되어 있습니다.

Backend & DevOps

기술	선정 이유 (Why?)
Vercel Serverless	별도의 백엔드 인프라 구축 없이 API 프록시를 구현하여 CORS 문제를 해결하고, 비용 효율적인 운영을 달성했습니다.
Supabase	Firebase의 대안으로, 관계형 데이터(매매 이력) 관리에 강점이 있는 PostgreSQL 기반의 BaaS를 선택했습니다.

4. 핵심 기능 (Key Features)

📊 1. 강력한 시뮬레이션 엔진 (Simulation Engine)

과거 1년치 데이터를 기반으로 사용자가 설정한 전략을 검증합니다.

- **전략 옵션:** 볼린저 밴드(BB), RSI, 이동평균선(MA), 거래량(Volume) 필터 조합.
- **자금 관리:** 마틴게일(Martingale) 및 역마틴게일 베팅 시스템 지원.
- **결과 분석:** 승률, 수익률, MDD(최대 낙폭), 거래 내역 등을 상세 리포트로 제공.

🧠 2. AI 기반 투자 분석 (AI Analytics)

최신 딥러닝 모델을 활용하여 단순한 기술적 분석의 한계를 보완합니다.

- **가격 예측 (TimesFM):** 시계열 파운데이션 모델을 활용하여 미래 가격 흐름 예측.
- **감성 분석 (FinBERT):** 주요 금융 뉴스 헤드라인을 분석하여 시장의 긍정/부정(Sentiment) 점수 산출.

⚡ 3. 실시간 매매 및 모니터링 (Real-time Trading)

- **자동 매매 (Auto Trading):** 장 마감 직전(또는 특정 시그널 발생 시) 자동으로 매수/매도 주문 실행.
- **포트폴리오 대시보드:** KIS API와 연동하여 내 계좌의 실시간 잔고, 수익률, 비중을 시각화.
- **실적 임팩트 분석:** 어닝 시즌(Earnings Call) 전후의 주가 변동성을 AI로 예측하여 리스크 관리.

5. 기술적 도전과 심층 분석 (Technical Deep Dive)

🔧 1. CORS 이슈와 Proxy 아키텍처

Situation: 브라우저에서 Yahoo Finance API를 직접 호출하면 No 'Access-Control-Allow-Origin' 오류가 발생. **Action:** 개발 환경에서는 Vite Proxy를, 프로덕션 환경에서는 Vercel Serverless Function을 사용하여 투명한 프록시 레이어를 구축했습니다. **Result:** 클라이언트는 동일 출처(/api/...)로 요청을 보내 CORS를 우회하며, API 키 등 민감 정보를 서버단에 은닉하여 보안성도 강화했습니다.

📦 2. 대용량 데이터 렌더링 최적화

Situation: 수년치 분봉 데이터나 수천 개의 거래 내역을 테이블에 렌더링할 때 DOM 노드 증가로 인한 FPS 저하 발생. **Action:**

- **가상화(Virtualization)** 도입: 화면에 보이는 영역만 렌더링하는 기법을 사용하여 메모리 사용량을 90% 절감.
 - **데이터 구조 최적화:** 차트 라이브러리에 전달하는 데이터를 필요한 필드(date, close)만 남기고 경량화.
- Result:** 데이터가 10,000건 이상이어도 스크롤 끊김 없는 부드러운 UX(60fps)를 달성했습니다.

🌐 3. 실시간 데이터 정합성 보장 (WebSocket + Polling)

Situation: WebSocket 연결이 일시적으로 끊기거나 패킷 손실이 발생할 경우, 잘못된 가격 정보를 기반으로 매매가 실행될 위험. **Action:** 하이브리드 동기화 모델을 설계했습니다. WebSocket으로 실시간 가격을 수신하되, 1분마다 REST API로 전체 캔들 데이터를 Polling하여 데이터 정합성을 검증(Double-Check)하고 보정합니다. **Result:** 네트워크 불안정 상황에서도 데이터 신뢰도 99.9%를 유지하며 안정적인 자동 매매 시스템을 구축했습니다.

6. 협업 및 코드 품질 (Quality & Collaboration)

1인 개발이지만 팀 프로젝트 수준의 코드 품질을 유지하기 위해 엄격한 규칙을 적용했습니다.

- **Commit Convention:** Udacity Git Style Guide와 Gitmoji를 결합하여 커밋 메시지의 가독성을 높였습니다.
- **Documentation:** JSDoc을 활용하여 주요 함수와 컴포넌트에 대한 문서를 코드 내에 포함시키고, README.md에 아키텍처를 시각화하여 유지보수성을 확보했습니다.
- **Thinking Process:** 단순히 기능을 구현하는 것을 넘어 '왜 이 기술을 쓰는가?', '더 나은 방법은 없는가?'를 끊임없이 고민하며 PORTFOLIO.md와 같은 기술 문서를 작성했습니다.

7. 프로젝트 회고 (Retrospective)

이 프로젝트를 통해 금융 데이터를 다루는 도메인 지식과 함께, **데이터의 수집(API) -> 가공(Logic) -> 시각화(UI) -> 자동화(Automation)**에 이르는 풀사이클 개발 역량을 길렀습니다. 특히 AI 모델을 웹 서비스에 통합하는 과정에서 MLOps의 기초를 경험할 수 있었으며, 앞으로도 "사용자에게 실질적인 가치를 주는 서비스"를 만드는 엔지니어가 되고자 합니다.